

Memory Management Mechanism — AgenticSys_v2

Snapshot: 2026-05-11. Reflects the codebase after commits 8ddce4e..fecdd3e (Phase 1–3 of the Memory-Management PRD, plus async distiller and chart-to-trace refinements).

1. Why this exists — the problem being solved

The case-review system slows down across long reviewer conversations because the orchestrator's `input_history` accumulates every turn's full specialist tool outputs (5–50 KB each), feeding 100–500 KB of context back to the model by turn 8–10. Two compounding deficiencies:

1. **No semantic recall across turns.** Specialists had only intra-orchestrator-run memory, so they re-ran identical `summarize_trend / aggregate_column` queries on every follow-up.
2. **Quantitative findings only as prose.** Reviewers find numeric trends and concentrations easier to parse as charts than narrative numbers.

The memory-management subsystem replaces unbounded raw-context replay with **structured, semantically-targeted recall**, paired with a routing signal so the orchestrator favors warm specialists on follow-ups.

2. The core idea (one paragraph)

Each specialist's findings are **distilled** into atomic, quantitative `KnowledgePoints` by a second-pass agent. These KPs are stored per-specialist in a session-scoped `specialist_kb`. On the specialist's next call, only the **active set** (latest per topic) is fed back as a short digest preface — so the specialist sees what it already knows without replaying the raw tool output. The orchestrator's `input_history` is pruned in parallel (old turns' tool-result payloads replaced by stubs), and a one-line **warmth hint** prepended to the user question tells the orchestrator which specialists are already loaded with knowledge for this case.

3. The four memory layers

The system maintains four distinct memory layers running in parallel, all scoped to a `CaseSession` and surviving across reviewer turns within that session.

```
CaseSession (server.py:67)

input_history — pruned every turn (keep last 2 turns intact)
specialist_kb — per-specialist list[KnowledgePoint], append-only
qa_cache — exact + near-duplicate Q→A cache
(warmth hint) — derived from specialist_kb, prepended each turn
```

↓
▼ per turn, threaded as `AppContext`

```
AppContext (agent_factories/app_context.py)

_specialist_kb → SAME DICT BY REFERENCE as session's KB
_specialist_histories → per-specialist intra-turn chat history
```

<code>_specialist_call_cache</code>	→ per-AppContext (name, sub_q) dedup
<code>_distiller</code>	→ the shared second-pass agent
<code>_pending_distillers</code>	→ fire-and-forget task handles
<code>_turn_id</code>	→ tagged onto each new KP

3.1 Orchestrator `input_history` — bounded conversational memory

- Field: `CaseSession.input_history` (server.py:80).
- Behavior: each turn appends user message + tool calls + tool outputs + final answer; replayed verbatim into `Runner.run_streamed` on the next turn.
- Bounding: `_prune_input_history` (server.py:238) runs after each turn.
 - Identifies turn boundaries by `{"role": "user", ...}` entries.
 - Keeps the last **2 reviewer turns** intact (`_INPUT_HISTORY_KEEP_RECENT_TURNS`).
 - In older turns, replaces `function_call_output.output` with the stub: `> "(elided - earlier-turn specialist output; see the specialist's KB digest, which is prepended to each new sub-question.)"`
 - Function-call records themselves are preserved □ orchestrator still sees *that* a tool ran, just not the heavy payload.
- Authoritative replay path for elided findings is the KB digest, **not** the history.

3.2 Per-specialist Knowledge Base — cross-turn semantic memory

- Field: `CaseSession.specialist_kb: dict[str, list[KnowledgePoint dict]]` (server.py:97).
- Shared by reference into each turn's AppContext. `_specialist_kb` so the `redacting_tool` wrapper's writes persist back to the session automatically.

KnowledgePoint schema (models/types.py:58):

field	type	purpose
topic	str (snake_case slug)	grouping key for supersession (latest per topic wins)
claim	str (1 sentence)	numbers + named entities + time window; faithfully extracted
numbers	list[dict]	data series behind the claim (trends, breakdowns, breaches)
viz	dict None	optional {kind, x_field, y_fields} chart spec
source_call	str	the tool invocation that produced the data
captured_at_turn	str None	short hex turn id — chronological audit
confidence	high medium low	reflects the specialist's hedging

Supersession model:

- The list is **append-only** — never mutated, never deleted (audit trail).
- `_active_kps` in `redacting_tool.py:36` iterates and keeps the **last-seen entry per topic**; this is what the specialist sees on its next call.
- Older entries with the same topic stay in the list so the logger can reconstruct what was believed at any point in the session.

Read path (redacting_tool.py:294):

- Only on the **first call** to a specialist within a turn, `_format_kb_digest` prepends a one-line-per-active-KP preface:

[YOUR KNOWLEDGE BASE – facts established earlier this session.
Refer to these BEFORE re-running queries; only re-query when the new question goes beyond what's recorded here, or when a value needs verification.]

– **monthly_spend_trend** [high]: Spend rose from \$300 (2024–11) to \$1,100 (2025–03)... _via `summarize\_trend('spends',...)`
– **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend (\$642K of \$

- Intra-turn follow-ups inherit the digest via the per-specialist conversation history (_specialist_histories), so re-prepending would duplicate.

Lifecycle: cleared by /rewind alongside input_history and qa_cache so a session reset wipes everything.

3.3 Async distiller — the KB writer

- Agent: agent_factories/distiller_agent.py — stateless, structured output (DistillerOutput □ knowledge_points[]), one shared instance per orchestrator.
- Built once in Orchestrator.__init__ (orchestrator/orchestrator.py:61) and exposed on AppContext._distiller.

Why a separate second pass instead of inline emission:

- Distillation is a different cognitive task than analysis. Asking the specialist to do both reliably bloats its prompt and degrades both.
- A narrowly-scoped agent with a strict output schema is more faithful (less paraphrasing) and cheaper to iterate.
- Failures degrade gracefully to “no KB update this turn” — the specialist’s answer is unaffected.

Strict extraction rules (from the distiller prompt):

- Faithful extraction only — every claim grounded directly in the SpecialistOutput; hedges preserved.
- Atomic — one quantitative fact per point; a 12-month trend series is ONE point, not 12.
- Quantitative bias — prefer numbers, named entities, comparisons; skip pure-narrative claims.
- Skip data-absence (already in SpecialistOutput’s data_gaps).

Fire-and-forget scheduling (redacting_tool.py:385):

After a specialist successfully returns:

```
task = asyncio.create_task(
    _distill_and_persist(app_ctx, name, redacted_in, result.final_output),
    name=f"distill-{name}",
)
app_ctx._pending_distillers.append(task)
return payload # orchestrator gets answer immediately
```

- Orchestrator receives the specialist’s payload **without** paying the distiller round-trip on the critical path.
- Server.py awaits all pending tasks at **end-of-turn** (server.py:1014, 60s budget) so the KB is fully populated before the next turn’s warmth digest is built.
- Per-distiller timeout: 30s (_DISTILLER_TIMEOUT_S). On timeout / error, logged as distiller_failed; the specialist answer is unaffected.

Chart rendering side-effect:

When a distilled KP carries viz + non-empty numbers, _distill_and_persist also:

- Calls kp_to_vega_spec(kp_dict) and stores the Vega-Lite v5 spec on the KP (kp_dict["vega_spec"] = spec).

- Calls `render_chart(kp_dict, charts_dir, turn_id=...)` to write `reports/<case_id>/charts/<turn_id>-<topic>.png`; stores the relative URL on `kp_dict["image_path"]`.
- Failures (matplotlib error, missing fields) are logged but never raised — KP still lands in the KB, just without a chart.

Charts surface in the reasoning-trace SSE panel (not inline in the chat answer) via `_collect_turn_charts` (`server.py:323`).

3.4 Warmth hint — routing signal for the orchestrator

- Built by `_format_kb_warmth_hint` (`server.py:298`).
- Format: `[KB-warmth: spend_payments (3 KPs), modeling (5 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]`
- Sorted by descending KP count; specialists with zero KPs are omitted (no “(0 KPs)” noise).
- Prepend to the redacted user question on every turn after the first (`server.py:684`):

```
framed_question = f"{warmth_hint}\n\n{verdict.redacted_question}"
```
- The `team_construction` skill is updated to treat this as a primary follow-up routing signal: “favor reusing warm specialists unless the question’s domain has clearly shifted.”
- **No hard skip** of team construction — the orchestrator retains full LLM judgment; the hint is *a signal*, not a programmatic bypass.
- Logged as `kb_warmth_hint_emitted` with `turn_id`, `warm_specialists`, `hint_length` (no PII, structural counts only).

4. Supporting caches

4.1 Per-specialist intra-turn history

- `AppContext._specialist_histories: dict[str, list]` (`app_context.py:22`).
- Updated by `redacting_tool` after each specialist run with `result.to_input_list()`.
- Lets a follow-up tool call to the same specialist **within the same AppContext** see what was already asked / answered, instead of starting fresh.
- Resets per-AppContext (i.e. per-turn).

4.2 Per-specialist call dedup

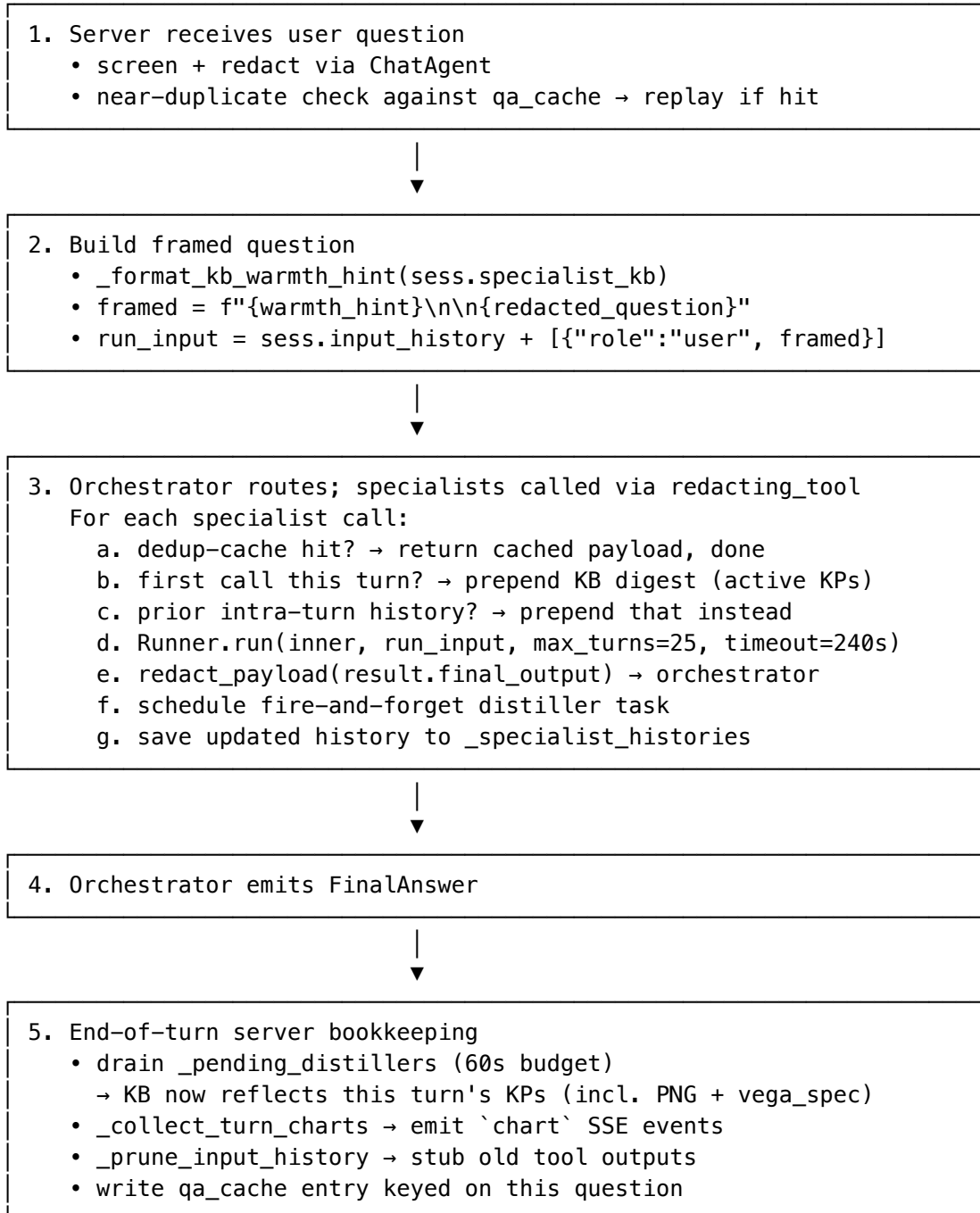
- `AppContext._specialist_call_cache: dict[tuple[name, normalized_subq], str]` (`redacting_tool.py:270`).
- Same `(specialist, normalized_sub_question)` within the same context returns the cached payload rather than re-running.
- Caps cost when the orchestrator (especially in safechain mode where parallel-tool-call semantics aren’t native) emits the same call multiple times in one turn with trivial wording variations.
- `_normalize_subq` collapses whitespace + lowercases.

4.3 Session QA cache

- `CaseSession.qa_cache: dict` (`server.py:89`).
- Keyed by `_normalize_q(redacted_question)`; value carries the cached `FinalAnswer` fields.
- **Exact-match** on the redacted reviewer question ☐ skips the orchestrator entirely on repeats.

- **Near-duplicate** path (server.py:589): ScreenVerdict.near_duplicate_of (computed by the relevance_check skill on subject + time-range + scope) re-keys into the cache for fuzzy matches.
 - Logged as qa_cache_hit / qa_cache_hit_near_duplicate.
-

5. End-to-end flow per turn



6. Worked example — three reviewer turns on one case

This walks through what happens in CaseSession state across three back-to-back turns on case C-00042, showing where each memory layer kicks in.

Turn 1 — cold start

Reviewer asks: *"What's the customer's spending pattern over the past 6 months?"*

Pre-run state:

```
sess.input_history    == []
sess.specialist_kb    == {}
sess.qa_cache         == {}
```

Framed question (no warmth hint — KB is empty):

What's the customer's spending pattern over the past 6 months?

Orchestrator routes □ calls spend_payments.

Specialist sees (no KB digest, no prior history — bare sub-question):

What is the customer's spending pattern over the past 6 months?

Specialist runs summarize_trend('spends', 'Amount', 'Date', period='month', op='sum') and aggregate_column('spends', 'Merchant', 'Amount', op='sum'), then returns a SpecialistOutput.

Fire-and-forget distiller kicks off after the redacted payload is returned to the orchestrator. It extracts:

```
[
  {
    "topic": "monthly_spend_trend",
    "claim": "Spend rose from $300 (2024-11) to $1,100 (2025-03), a 3.7× increase peaking in 2025-03",
    "numbers": [
      {"period": "2024-11", "value": 300},
      {"period": "2024-12", "value": 250},
      {"period": "2025-01", "value": 480},
      {"period": "2025-02", "value": 920},
      {"period": "2025-03", "value": 1100},
      {"period": "2025-04", "value": 760}
    ],
    "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
    "source_call": "summarize_trend('spends','Amount','Date',period='month',op='sum')",
    "captured_at_turn": "a3f7c1e9d2b4",
    "confidence": "high"
  },
  {
    "topic": "top_merchants_by_sum",
    "claim": "S BERTRAM accounts for 38% of recurring spend ($642K of $1.69M total).",
    "numbers": [
      {"group": "S BERTRAM", "value": 642000},
      {"group": "Other", "value": 1052000}
    ],
    "viz": null,
    "source_call": "aggregate_column('spends','Merchant','Amount',op='sum')",
    "captured_at_turn": "a3f7c1e9d2b4",
    "confidence": "medium"
  }
]
```

```
}
]
```

The first KP has viz.kind=trend with 6 numbers. The renderer writes reports/C-00042/charts/a3f7c1e9d2b4-monthly_spend_trend.png and attaches the Vega-Lite spec.

End-of-turn drain awaits the distiller task. Post-turn state:

```
sess.specialist_kb == {
  "spend_payments": [
    { ... monthly_spend_trend KP, with image_path + vega_spec ... },
    { ... top_merchants_by_sum KP ... },
  ]
}
sess.input_history == [<6 items: user msg + tool calls + outputs + final answer>]
sess.qa_cache      == {"what's the customer's spending pattern over the past 6 months":
                       <FinalAnswer payload>}
```

A chart SSE event is emitted for the trend chart; reviewer sees it in the reasoning-trace panel.

Turn 2 — warm follow-up, same domain

Reviewer asks: “Did the spending peak coincide with any payment returns?”

Pre-run state: KB has 2 KPs under spend_payments; the QA cache holds turn 1.

Warmth hint built by _format_kb_warmth_hint:

[KB-warmth: spend_payments (2 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]

Framed question (warmth hint prepended):

[KB-warmth: spend_payments (2 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]

Did the spending peak coincide with any payment returns?

Orchestrator reads the warmth hint, applies the team_construction rule, and reuses spend_payments rather than building a fresh team.

Specialist sees on its first call this turn (KB digest preface — _active_kps returns latest-per-topic):

[YOUR KNOWLEDGE BASE — facts established earlier this session.
Refer to these BEFORE re-running queries; only re-query when the new question goes beyond what's recorded here, or when a value needs verification.]

— **monthly_spend_trend** [high]: Spend rose from \$300 (2024-11) to \$1,100 (2025-03), a 3.7× increase peaking in 2025-Q1. _via `summarize\_trend('spends', 'Amount', 'Date', period='1Y')`
— **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend (\$642K of \$1.69M)

--- New question ---

Did the spending peak coincide with any payment returns?

The specialist now knows the peak is 2025-Q1 without re-running summarize_trend('spends'). It runs only the *new* call:

```
filter_rows('payments', where='status == "returned"', group_by='month')
```

It returns a SpecialistOutput referencing the existing peak and the new returns timeline.

Distiller adds one new KP:

```
{
  "topic": "payment_returns_timeline",
  "claim": "Payment returns spiked from 0 (2024-Q4) to 4 (2025-Q1) coinciding with the spend p",
  "numbers": [
    {"period": "2024-Q4", "value": 0},
    {"period": "2025-Q1", "value": 4},
    {"period": "2025-Q2", "value": 1}
  ],
  "viz": null,
  "source_call": "filter_rows('payments', where='status == \"returned\"', group_by='month')",
  "captured_at_turn": "b7e2d4f8a1c6",
  "confidence": "high"
}
```

(numbers has only 3 entries — distiller's rule says viz only when ≥ 4 , so no chart this turn.)

Post-turn state:

```
sess.specialist_kb == {
  "spend_payments": [
    monthly_spend_trend          # turn 1
    top_merchants_by_sum,        # turn 1
    payment_returns_timeline,    # turn 2 ← new
  ]
}
```

input_history now has both turns; pruning hasn't triggered yet (keep_recent_turns=2).

Turn 3 — cross-domain question, supersession, history pruning

Reviewer asks: "How does that align with their FICO trajectory? Also, what does spending look like if we extend to 12 months?"

Warmth hint:

[KB-warmth: spend_payments (3 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]

Orchestrator decides this needs bureau (cold) for FICO + spend_payments (warm) for the extended trend. Because two domain specialists run, general_specialist is also invoked (cross-domain review protocol).

bureau specialist is called first. No KB digest (its KB is empty). It runs FICO history queries and returns. Distiller extracts:

```
{
  "topic": "fico_trajectory",
  "claim": "FICO declined from 712 (2024-10) to 648 (2025-04), a 64-point drop steepest in 202",
  "numbers": [
    {"period": "2024-10", "value": 712}, {"period": "2024-11", "value": 708},
    {"period": "2024-12", "value": 695}, {"period": "2025-01", "value": 671},

```

```

    {"period": "2025-02", "value": 658}, {"period": "2025-03", "value": 651},
    {"period": "2025-04", "value": 648}
  ],
  "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
  "source_call": "summarize_trend('bureau','FicoScore','Date',period='month',op='last')",
  "captured_at_turn": "c8a3f2e5d917",
  "confidence": "high"
}

```

numbers has 7 entries + viz is set □ renderer writes the chart PNG.

spend_payments specialist is called for the 12-month extension. It sees the digest (now 3 KPs) and recognizes that `monthly_spend_trend` only covered 6 months. It re-queries with a wider window and returns a new `SpecialistOutput`.

Distiller extracts a NEW `monthly_spend_trend` KP (same topic, wider window). The KB list grows to 4 entries under `spend_payments` — but `_active_kps` will hide the older 6-month version on the next call:

```

# After this turn's distillation:
sess.specialist_kb["spend_payments"] == [
    monthly_spend_trend_v1,      # turn 1 – kept for audit, hidden from digest
    top_merchants_by_sum,        # turn 1
    payment_returns_timeline,    # turn 2
    monthly_spend_trend_v2,      # turn 3 – supersedes v1 in the active view
]
# _active_kps(...) returns [top_merchants_by_sum, payment_returns_timeline, monthly_spend_trend_v2]

```

input_history pruning (this is now the third reviewer turn □ turn 1 ages out of the “keep recent” window):

- All `function_call_output.output` strings from turn 1 are replaced with the elision stub: > "(elided – earlier-turn specialist output; see the specialist's KB digest, which is prepended to each new sub-question.)"
- Function-call records themselves stay (orchestrator still sees what tools ran).
- Net effect: maybe 40 KB of raw `SpecialistOutput` JSON replaced by ~120 bytes of stub × N calls.

`input_history_pruned` event fires with `bytes_saved` reported.

Two charts are emitted as SSE events this turn (`fico_trajectory` + `monthly_spend_trend_v2`); the reviewer sees both in the reasoning-trace panel.

What the example demonstrates

layer	shown by
KB digest preface	Turn 2: specialist knew the peak was Q1 without re-querying.
Supersession	Turn 3: <code>monthly_spend_trend_v2</code> shadows <code>v1</code> in the active digest; <code>v1</code> stays in the list for audit.
Warmth hint as routing input	Turn 2: orchestrator reused the warm specialist; Turn 3: still considered both, picked correctly per question.
Fire-and-forget distiller	Specialist payloads returned to orchestrator immediately; KB updates landed at end-of-turn drain.
Chart rendering side-effect	Turns 1 + 3 wrote chart PNGs + Vega-Lite specs; Turn 2 didn't (only 3-point series).
<code>input_history</code> pruning	Turn 3: turn 1's heavy tool outputs replaced by stubs.

layer	shown by
QA cache	If the reviewer re-asks the Turn 1 question verbatim, the orchestrator is skipped entirely.
Dedup cache	Would kick in if the orchestrator emitted two near-identical sub-questions to spend_payments in one turn.

7. Design trade-offs (the “why” behind the shape)

decision	rationale
Second-pass distillation instead of inline KP emission	Specialist + distiller are different cognitive tasks; combining them bloats prompts and degrades faithfulness. Separate agent is also cheaper to iterate.
Fire-and-forget distiller	The orchestrator should not pay the distiller round-trip on the critical path. The KB only needs to be fresh for the <i>next</i> turn, so end-of-turn drain is sufficient.
Append-only KB with implicit supersession	Audit trail is preserved (operators can reconstruct what was believed at any turn) while the active view stays small.
Short digest preface, not full KB replay	The job is to <i>prevent re-querying</i> , not to replay every detail. The specialist can still re-query when verification is needed.
Warmth hint as soft signal, not hard short-circuit	Keeps team construction LLM-driven so the orchestrator can override on clear domain shifts. Programmatic bypass would be a separate exact-near-duplicate fast-path.
input_history pruning preserves call records	Orchestrator still sees <i>that</i> a tool was invoked (avoids spurious re-calls), but elides the heavy payload (saves tokens).
Charts to reasoning-trace panel, not chat answer	Keeps the chat clean (text only); reviewers get click-to-open access tied to the specific finding. Vega-Lite spec is preserved for future interactive UIs.
No RAG over KB	Digest is passed verbatim; embedding-based retrieval is deferred until KBs grow past ~50 entries per specialist (not observed in current sessions).
Domain specialists see only their own KB	Cross-domain sharing (e.g. through <code>general_specialist</code>) is intentionally deferred until use cases firm up.

8. Observability hooks

The EventLogger emits the following memory-related events:

event	trigger	payload (structural only — no PII)
kb_warmth_hint_emitted	non-empty hint built before a turn	turn_id, warm_specialists [{name, n_kps}], hint_length
distiller_kps_added	distiller produced ≥ 1 KP	specialist, n_added, kb_size_now, topics[], n_with_charts
distiller_failed	distiller run errored or timed out	specialist, error_type, error_message (truncated)
distiller_drain_timeout	end-of-turn drain exceeded 60s	turn_id, n_pending
distiller_outer_failure	task creation itself failed (rare)	specialist, error_type, error_message

event	trigger	payload (structural only — no PII)
input_history_pruned	history pruning produced any elisions	counts of items elided, bytes saved
specialist_call_deduped	dedup cache served a call	specialist, sub_question_norm
qa_cache_hit / qa_cache_hit_near_duplicate	session-level QA cache served the private	turn_id, cached_question (redacted)
qa_cache_store	new answer cached after a turn	structural counts

9. Known limits / explicit non-goals

- **No interactive client-side charts** — Phase 2 ships static PNG + Vega-Lite spec; interactive UI is future work.
- **No hard-skip routing** — Phase 3 deliberately keeps team construction as an LLM decision.
- **No RAG / embedding retrieval over KB** — deferred until KBs grow past ~50 entries.
- **No cross-specialist KB sharing** (except future general_specialist).
- **No chart re-rendering for historical KPs** — only KPs captured in the current turn render charts; pre-Phase-2 KPs aren't back-filled.
- **No “reset case” admin path** — /rewind wipes session state but chart files persist on disk for audit.

10. Files of interest

concern	file
Session state, warmth, pruning	server.py (CaseSession, _prune_input_history, _format_kb_warmth_hint)
KB read path + distiller scheduling	agent_factories/redacting_tool.py
Per-turn context plumbing	agent_factories/app_context.py
Distiller agent + prompt	agent_factories/distiller_agent.py
KnowledgePoint schema	models/types.py
Distiller construction	orchestrator/orchestrator.py
Chart rendering	tools/viz_renderer.py
Routing skill (consumes warmth)	skills/workflow/team_construction.md
Original PRD	tasks/prd-memory-management.md